El Lenguaje



Herencia

Herencia en C++

- *La herencia nos permite reutilizar el código de proyectos previos.*
- *Tiene la flexibilidad suficiente de permitirnos modificar ciertas características, si es necesario, sin tocar el código fuente.*
- *Esto es muy importante porque no necesitamos realizar depuración sobre lo que ya está construido por el hecho de introducir modificaciones.*

Herencia en C++

- *En todo caso solo debemos probar lo nuevo.*
- *La herencia es una de las principales ventajas del trabajo con clases.*
- *Implica una "especialización", ya que conservamos todas las propiedades de la clase original agregando las particu-laridades de la nueva clase que esta-mos definiendo.*

Herencia en C++

- *Si una clase hereda de otra se denomina: clase derivada y su ancestro: clase base.*
- *A su vez esta clase derivada puede ser clase base de otras.*
- *Mediante este mecanismo podemos generar una jerarquía de clases tan compleja como querramos.*

Herencia en C++

- *Por ejemplo si definimos una clase mamíferos, contemplamos en ella las características que comparten TODOS los mamíferos.*
- *De ella se deriva la clase "humanos", que agrega a las características de los mamíferos las propiedades que la diferencian del resto.*
- *Igual sucedería con una clase "niños".*

Herencia en C++

- *La herencia en la programación orientada a objetos es reflejo de un aspecto fundamental de la inteligencia humana: buscar, reconocer y crear relaciones entre conceptos.*
- *Una clase derivada tiene menos integrantes y más detalles (datos y funciones, esto es, complejidad) que la clase base.*

Revisando tipos de acceso

```
class complejo{
  private:
    double re, im;
  public:
    complejo( double preal, double pimag );
    print();
  protected:
    int signo();
};
```

- *Cuando vimos la declaración de una clase teníamos los tipos de acceso private y public.*

Revisando tipos de acceso

- *Las clases derivadas no tienen acceso a los miembros privados de la clase base.*
- *Ahora agregamos: protected.*
- *Para la clase base, protected se comporta como private.*
- *Las funciones de las clases derivadas tienen acceso a los miembros public y protected de la clase base.*

Declaración de clase heredada

- *La sentencia es la siguiente:*

```
class <c_d> : [public|protected|private] <c_b>
// c_d: clase derivada                  c_b: clase base
```

- *Vemos que se puede heredar en forma pública, protegida o privada.*
- *Si se hereda en forma public (lo más común), permite que sus métodos y objetos accedan a los datos protegidos y públicos de la clase base.*

Declaración de clase heredada

- *La herencia "public" tiene el significado de "es un".*

  Ej.: `class canario : public ave { .... };`

- *Si se hereda en forma private, se inhibe que las clases derivadas de esta última puedan acceder a los miembros de la clase base (la original).*

Declaración de clase heredada

- *Si se hereda en forma protegida, se sigue sosteniendo para la clase heredante el significado de los datos privados, protegidos y públicos.*
- *El resultado es para las clases que se deriven de la actual.*
- *Hay que ser muy cuidadoso con la forma de heredar, comprendiendo porqué se deriva de una determinada manera.*

```
#include <iostream>  // P04-01.cpp
using namespace std;
class ClaseA {
  public:
    ClaseA(int a=10) : datoA(a) {cout << "Constructor de A" << endl;}
    int LeerA() { return datoA; }
  protected:
    int datoA;
};

class ClaseB : public ClaseA {
  public:
    ClaseB(int a=10, int b=20) : ClaseA(a), datoB(b)
      {cout << "Constructor de B" << endl;}
    int LeerB() { return datoB; }
  protected:
    int datoB;
};

int main()
{
  ClaseB objeto1;
  cout << "datoA = " << objeto1.LeerA() <<
          ", datoB = " << objeto1.LeerB() << endl;
  ClaseB objeto2( 100, 200);
  cout << "datoA = " << objeto2.LeerA() <<
          ", datoB = " << objeto2.LeerB() << endl;
  return 0;
}
```

```
Salida:
Constructor de A
Constructor de B
datoA = 10, datoB = 20
Constructor de A
Constructor de B
datoA = 100, datoB = 200
```

Declaración de clase heredada

- *En el ejemplo anterior si datoA se hubiera declarado private, ClaseB no hubiera tenido acceso, dando un error.*
- *En realidad ClaseB no hace acceso directo a datoA, solo utiliza el constructor.*
- *Observar cómo pasa el dato el constructor de ClaseB a ClaseA para que este inicialice la parte que le corresponde.*

```
#include <iostream>  // P04-02.cpp
using namespace std;
class ClaseA {
  public:
    ClaseA(int a=10) : datoA(a)
      {cout << "Constructor de A" << endl;}
    int LeerA() { return datoA; }
  protected:
    int datoA;
};

class ClaseB : public ClaseA {
  public:
    ClaseB(int a=10, int b=20) : ClaseA(a), datoB(b)
      {cout << "Constructor de B" << endl;}
    int LeerB() { return datoB; }
    void incAB() { datoA++; datoB++; }
  protected:
    int datoB;
};
```

- *En este ejemplo el objeto modifica los datos de su clase directa y también de su antecesora.*

```
int main()
{
  ClaseB objeto1;
  cout << "datoA = " << objeto1.LeerA() <<
          ", datoB = " << objeto1.LeerB() << endl;
  ClaseB objeto2( 100, 200);
  cout << "datoA = " << objeto2.LeerA() <<
          ", datoB = " << objeto2.LeerB() << endl;
  objeto2.incAB();
  cout << "datoA = " << objeto2.LeerA() <<
          ", datoB = " << objeto2.LeerB() << endl;
  return 0;
}
```

```
Salida:
Constructor de A
Constructor de B
datoA = 10, datoB = 20
Constructor de A
Constructor de B
datoA = 100, datoB = 200
datoA = 101, datoB = 201
```

Declaración de clase heredada

- *En el caso siguiente, declaramos:*

```
private:
  int datoA;
```

- *El programa no compilará porque estamos tratando de acceder un miembro dato que ha sido declarado como private.*

```
#include <iostream.h>  // P04-03.cpp

using namespace std;

class ClaseA {
  public:
    ClaseA(int a=10) : datoA(a)
      {cout << "Constructor de A" << endl;}
    int LeerA() { return datoA; }
  private:
    int datoA;
};

class ClaseB : public ClaseA {
  public:
    ClaseB(int a=10, int b=20) : ClaseA(a), datoB(b)
      {cout << "Constructor de B" << endl;}
    int LeerB() { return datoB; }
    void incAB() { datoA++; datoB++; }
  protected:
    int datoB;
};
```

```
Compilación:
P04-03.cpp: In member function `void ClaseB::incAB()':
P04-03.cpp:10: `int ClaseA::datoA' is private
P04-03.cpp:18: within this context
```

```
int main()
{
  ClaseB objeto1;
  cout << "datoA = " << objeto1.LeerA() <<
          ", datoB = " << objeto1.LeerB() << endl;
  ClaseB objeto2( 100, 200);
  cout << "datoA = " << objeto2.LeerA() <<
          ", datoB = " << objeto2.LeerB() << endl;
  objeto2.incAB();
  cout << "datoA = " << objeto2.LeerA() <<
          ", datoB = " << objeto2.LeerB() << endl;
  return 0;
}
```

```
Compilación:
P04-03.cpp: In member function `void ClaseB::incAB()':
P04-03.cpp:10: `int ClaseA::datoA' is private
P04-03.cpp:18: within this context
```

Herencia "private"

- *Veamos el caso siguiente en que tenemos una clase que hereda en forma privada.*
- *Esto significa que los miembros protegidos y públicos serán visibles por los métodos de la clase derivada, pero no directamente por los objetos de la clase derivada.*

```
#include <iostream>  // P04-04.cpp
using namespace std;
class ClaseA {
  public:
    ClaseA(int a=10) : datoA(a) {cout << "Constructor de A" << endl;}
    int LeerA() { return datoA; }
  protected:
    int datoA;
};

class ClaseB : private ClaseA {
  public:
    ClaseB(int a=10, int b=20) : ClaseA(a), datoB(b)
      {cout << "Constructor de B" << endl;}
    int LeerB() { return datoB; }
  protected:
    int datoB;
};


int main()
{
  ClaseB objeto;
  cout << "datoA = " << objeto.LeerA() <<
          ", datoB = " << objeto.LeerB() << endl;
  return 0;
}
```

Compilación:

```
P04-04.cpp: In function 'int main()':
P04-04.cpp:8: error: 'int ClaseA::LeerA()' is inaccessible
P04-04.cpp:26: error: within this context
P04-04.cpp:26: error: 'ClaseA' is not an accessible base of
'ClaseB'
```

Herencia "private"

- *En el ejemplo siguiente vemos que pueden acceder a los datos públicos y protegidos de la clase base los métodos de la clase derivada.*
- *Si la jerarquía de clases sigue, hacer esta herencia privada impedirá que las clases derivadas de esta última tengan acceso a los miembros de la primera clase base.*

```
#include <iostream.h>  // P04-05.cpp

using namespace std;

class ClaseA {
  public:
    ClaseA(int a=10) : datoA(a) {cout << "Constructor de A" << endl;}
    int LeerA() { return datoA; }
  protected:
    int datoA;
};

class ClaseB : private ClaseA {
  public:
    ClaseB(int a=12, int b=22) : ClaseA(a), datoB(b)
      {cout << "Constructor de B" << endl;}
    int LeerB() { return datoB; }
    void LeerT() { cout << "datoA = " << datoA <<
                   ", datoB = " << datoB << endl; }
    void incAB( int a, int b ) { datoA += a; datoB += b; }
  protected:
    int datoB;
};

int main() {
  ClaseB objeto;
  objeto.LeerT();
  objeto.incAB( 7, 8 );
  cout << "Después del incremento" << endl;
  objeto.LeerT();
  return 0;
}
```

Salida:

```
Constructor de A
Constructor de B
datoA = 12, datoB = 22
Después del incremento
datoA = 19, datoB = 30
```

Combinaciones "private/protected"

- *Es importante reconocer las implican-cias de esta forma de herencia.*
- *Un tema es la mecánica, esto es, qué se puede y qué no se puede.*
- *Otro es cuál es el significado de adoptar una forma sobre la otra.*
- *En el ejemplo 4-6 se puede ver cuáles son los alcances de cada una.*

es un... / tiene un...

- *Más allá de las cuestiones sintácticas es importante comprender el significado.*
- *La herencia en su plenitud es la que se realiza en forma pública, la que significa: "es un...", nadie puede renegar de su origen y se identifica en forma completa con este.*
- *"tiene un.." es una característica de conveniencia que una clase incorpora.*

es un... / tiene un...

- *Esto es lo que implica la herencia privada o protegida.*
- *La herencia privada puede ser reemplazada por clases que contienen otras clases.*
- *La herencia privada es válida, pero su significado es totalmente distinto, acá se aprovecha un código escrito sin identificarse completamente con el origen.*

- *Verificar el código siguiente:*

```
class Persona { };  // P04-07.cpp

class Estudiante:
  private Persona { };

void bailar(const Persona& p) {}      // cualquiera baila

void estudiar(const Estudiante& s) {} // solo los estudiantes
                                      // estudian


int main( void ) {
  Persona p;                          // p es una Persona
  Estudiante s;                       // s es un Estudiante

  bailar(p);                          // está bien, p is una
                                      // Persona


  bailar(s);                          // error! un Estudiante no es
                                      // una Persona
}
```

Compilarlo también cambiando private por public

```
Compilación:
P04-07.cpp: In function 'int main()':
P04-07.cpp:16: error: 'Persona' is an inaccessible base of
'Estudiante'
```

Clases que tienen clases

- *¿Cuándo definir clases que tienen clases adentro? ¿o herencia privada? (resultan bastante similares).*
- *La respuesta es: usar clases que tienen clases todas las veces que se pueda, usar herencia privada (o protegida) las veces que se deba.*
- *Esto último especialmente cuando tenemos funciones virtuales.*

Construcción de Jerarquías

- *Es el principio de la programación orientada a objetos.*
- *Prácticamente todo a nuestro alrededor son clasificaciones de distintas formas que constituyen las jerarquías más variadas.*
- *Clasificar es el punto de partida para buscar la solución de cualquier problema*

Construcción de Jerarquías

- *Esto es más evidente cuando los problemas son complejos.*
- *Tal como casi todo lo que encaramos en la vida real.*
- *Un objeto que hereda de varias clases en general es un elemento complejo que tiene las características de todos los antecesores, más las propias.*

Construcción de Jerarquías

- *Siempre podemos crear nuevas clases para resolver situaciones nuevas.*
- *Debemos ser muy cuidadosos en los a-tributos, observando bien si un atributo es particular a la clase o corresponde a un antecesor.*
- *La buena construcción de jerarquías es algo que se aprende con la experiencia.*

Construir bien la Jerarquía

- *La programación orientada a objetos tiende a acercarse a la forma en que pensamos.*
- *Esto produce a veces que nuestros e-rrores se trasladen a la definición de je-rarquías y nos lleva a la conclusión de que "el método falla".*
- *No es así, debemos revisar bien.*

Construir bien la Jerarquía

- *Ejemplo*

```
class Pajaro {
   public:
      void vuelo(); // los pájaros vuelan
   ...
};
class Pinguino : public Pajaro { // los pingüinos son
...                              // pájaros
};
```

- *¿Qué falló?*

Construir bien la Jerarquía

- *Falla la concepción de la jerarquía, no todos los pájaros vuelan.*
- *Por más que mucha gente crea que ese es el distintivo de todos los pájaros.*
- *Esto conduce a que debamos definir bien la clase base y también las clases derivadas.*
- *Quedaría de la forma siguiente.*

```
class Pajaro {
  ...                 // ¡no se declara función de vuelo!
};

class PajaroVolador : public Pajaro {
  public:
    void vuelo();
  ...
};

class PajaroNoVolador : public Pajaro {
  ...                 // no se declara función de vuelo
};

class Pinguino : public PajaroNoVolador {
  ...
};
```

- *Queda restablecida la lógica...*

Herencia Múltiple

- *C++ permite la creación de clases derivadas a partir de varias clases base.*
- *Los objetos creados a partir de estas clases, heredarán los datos y funciones de todas las clases base.*
- *La sentencia de derivación es así:*

```
<clase_derivada>(<lista_de_parámetros>) :
<clase_base1>(<lista_de_parámetros>)[,
<clase_base2>(<lista_de_parámetros>)] {}
```

Herencia Múltiple

- *Hay que ser muy cuidadoso con la herencia múltiple, ya que puede haber problemas, como por ejemplo que existan funciones miembro con el mismo nombre.*
- *Esto generará ambigüedad cuando se invoca a una de estas funciones.*
- *Veamos el ejemplo siguiente:*

```
#include <iostream.h>  // P04-08.cpp

using namespace std;

class ClaseA {
  public:
    ClaseA() : valorA(10) {}
    int LeerValor() const {return valorA;}
  protected:
    int valorA;
};

class ClaseB {
  public:
    ClaseB() : valorB(20) {}
    int LeerValor() const {return valorB;}
  protected:
    int valorB;
};

class ClaseC : public ClaseA, public ClaseB {};
```

- *Observar el nombre* `LeerValor()`

```
int main()
{
  ClaseC CC;
  cout << CC.LeerValor() << endl;
  // Produce un error de compilación ya que hay
  // ambigüedad.
  // cout << CC.ClaseA::LeerValor() << endl;
  return 0;
}
```

```
Compilación:
P04-08.cpp: In function `int main()':
P04-08.cpp:24: request for member `LeerValor' is ambiguous
P04-08.cpp:14: candidates are: int ClaseB::LeerValor() const
P04-08.cpp:6:                  int ClaseA::LeerValor() const
```

- *Para salvar esta ambigüedad se debe utilizar la sentencia comentada en lugar de la que está activa.*

- *La redefinición de funciones en la herencia permite salvar la situación de otra manera.*
- *Este aspecto también muestra una de las posibilidades más importantes de la herencia.*
- *Esto es, redefinir datos y funciones, para evitar efectos indeseados o inapropiados al comportamiento general.*
- *Veamos cómo se realiza en el ejemplo siguiente:*

```
#include <iostream.h>  // P04-09.cpp

using namespace std;

class ClaseA {
  public:
    ClaseA() : valorA(10) {}
    int LeerValor() const { return valorA; }
  protected:
    int valorA;
};

class ClaseB {
  public:
    ClaseB() : valorB(20) {}
    int LeerValor() const { return valorB; }
  protected:
    int valorB;
};

class ClaseC : public ClaseA, public ClaseB {
  public:
    int LeerValor() const {return ClaseA::LeerValor();}
};
```

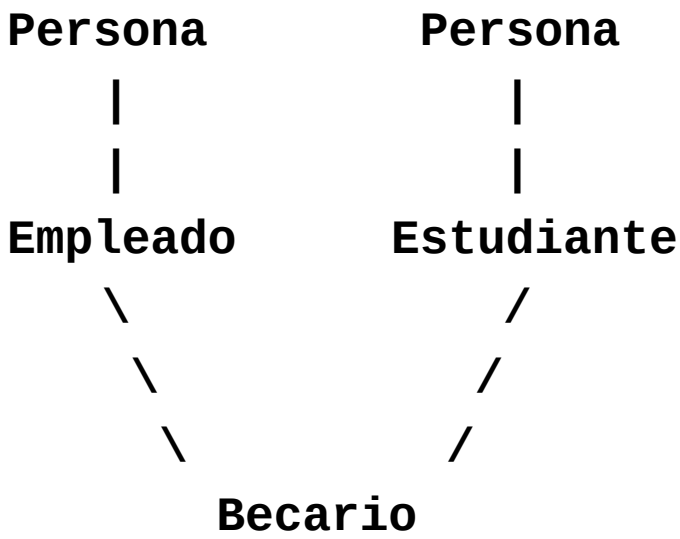
```
Salida:
10
20
```

```
int main() {
  ClaseC CC;
  cout << CC.LeerValor() << endl;
  cout << CC.ClaseB::LeerValor() << endl;
  return 0;
}
```

- *Ahora cuando se dice LeerValor() corresponde a la redefinición de esta función que se hizo en la ClaseC.*
- *Si se quiere acceder a la que suministra la ClaseB hay que hacerlo de la misma forma que en el ejemplo anterior.*

Herencia Virtual

- *Si tenemos una estructura de datos como la que se indica a continuación:*



Herencia Virtual

- *La clase becario heredará dos veces los datos y funciones de la clase Persona.*
- *Esto provoca ambigüedad a la hora de acceder a los datos y funciones hereda-das de la clase Persona.*
- *Para solucionar esto se usan las clases virtuales, mediante herencia virtual.*

Herencia Virtual

- *Declaración de herencia virtual:*

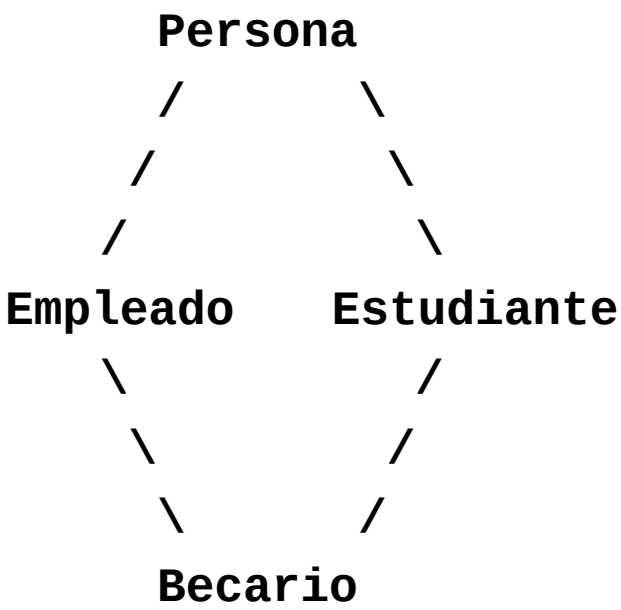
```
class Estudiante : virtual public Persona {};
```

- *La declaración completa quedaría así:*

```
class Estudiante : virtual public Persona {};
class Empleado : virtual public Persona {};
class Becario : public Estudiante, public Empleado {};
```

Herencia Virtual

- *Ahora la estructura queda así:*



Herencia Virtual

- *Debe tenerse un cuidado especial con los constructores, los que deben ser convocados desde la última clase derivada.*
- *En el caso de herencia virtual estos no son convocados automáticamente.*
- *En el ejemplo que sigue podemos ver cómo es el proceso.*

```
#include <iostream.h>  // P04-10.cpp
#include <string.h>

using namespace std;

class Persona {
  public:
    Persona(char *n) { strcpy(nombre, n); }
    const char *LeeNombre() { return nombre; }
  protected:
    char nombre[30];
};

class Empleado : virtual public Persona {
  public:
    Empleado(char *n, int s) : Persona(n), salario(s) {}
    int LeeSalario() const { return salario; }
    void ModificaSalario(int s) { salario = s; }
  protected:
    int salario;
};
```

```
class Estudiante : virtual public Persona {
  public:
    Estudiante(char *n, float no) : Persona(n), nota(no) {}
    float LeeNota() const { return nota; }
    void ModificaNota(float no) { nota = no; }
  protected:
    float nota;
};

class Becario : public Empleado, public Estudiante {
  public:
    Becario(char *n, int s, float no) :
      Empleado(n, s), Estudiante(n, no), Persona(n) {}
};

int main() {
  Becario Admin1("Juan A. Pérez", 1000, 7);
  cout << "Becarios de Administración: " << endl;
  cout << Admin1.LeeNombre() << "," << Admin1.LeeSalario()
  << "," <<
  Admin1.LeeNota() << endl;
  return 0;
}
```

Salida:
Becarios de Administración:
Juan A. Pérez,1000,7